

1. Provide a high-level summary of the logic used to identify p for each iteration:
(note all high-level logic is written in 0-based indexing as it is coded in)

The algorithm is driven by the function `Yet_another_pattern_matching_algorithm`, that finds all valid occurrences of some string pattern in a text. As specified in the assignment the algorithm aligns the pattern against the text at some position j starting from the right end of the text and shifting leftwards ($j = n - m$, where j is strictly decreasing).

Implemented to improve time complexity is the Galil rule to skip re-comparing characters after known matches have been found after a shift.

To derive p , which is used to compute the shift value of the algorithm, a $(m+1)$ by $|\Sigma|$ shift lookup table is preprocessed. This table “combines” the Good suffix rule and Bad character rule of the Boyer-Moore algorithm into an instant look up structure. Logically this is done via noting for any mismatch occurring at position k in the pattern, a **match region exists**, $\alpha = \text{pat}[k+1 \dots m]$ that matched the text, and the text character at the mismatch is **$\text{txt}[j+k]$** , character **x** in the assignment diagram.

The table then stores: $\text{shift_table}[k+1][c] = p$. Where p is the leftmost position such that $\text{pat}[p \dots m] = \text{pat}[k+1 \dots k+m-p]$ and $\text{pat}[p-1] = \text{txt}[j+k-1]$ where $\text{txt}[j+k-1] = c$ or **x** .

To build this table I used a border chain inspired logic from the KMP algorithm. (Further researched online: <https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/>) The algorithm first computes a list that defines the length of the longest prefix of the pattern from the to some point i , that is also a suffix of the pattern. This is stored at $\text{lps}[i]$ where the prefix cannot be the entire string.

The value at $\text{LPS}[m-1]$ also gives the safest shift after a full pattern match since it defines the entire pattern as a border, and maintains prefix suffix matching.

From this we compute the same structure on the reversed pattern, giving us at each index i , the longest border of the reversed suffix, which is the longest border of the original suffix at $\text{pat}[m-i-1 \dots m]$. This means for each index k in the pattern, $\text{pat}[k \dots m]$ is mapped to $\text{Rev_lps}[m-k-1]$.

We now populate the shift table via traversing this reversed LPS from $m-1 \dots 1$. Firstly we initialize all values to $m+1$, to represent the case of the default value of p .

Then as we iterate we set $\text{table}[i][c] = m - \text{rev_lps}[i]$, where c is the character to the left of a prefix-suffix match, formally $c = \text{Pat}[\text{rev_lps}[i] - 1]$.

Here $M - \text{rev_lps}[i]$ is the index of the start of the suffix that matches a prefix for any window of text, this is p .

That is to say we identify P for any index i that defines the longest prefix-suffix match for any given substring from right to left of our pattern that also satisfies the character to the left = the mismatched character from the text.

If rev_lps returns 0, there is no matched suffix so we can shift by the entire pattern for any defined character, letting $p = m$.

2. Provide a formal or logical proof demonstrating that your implemented shift

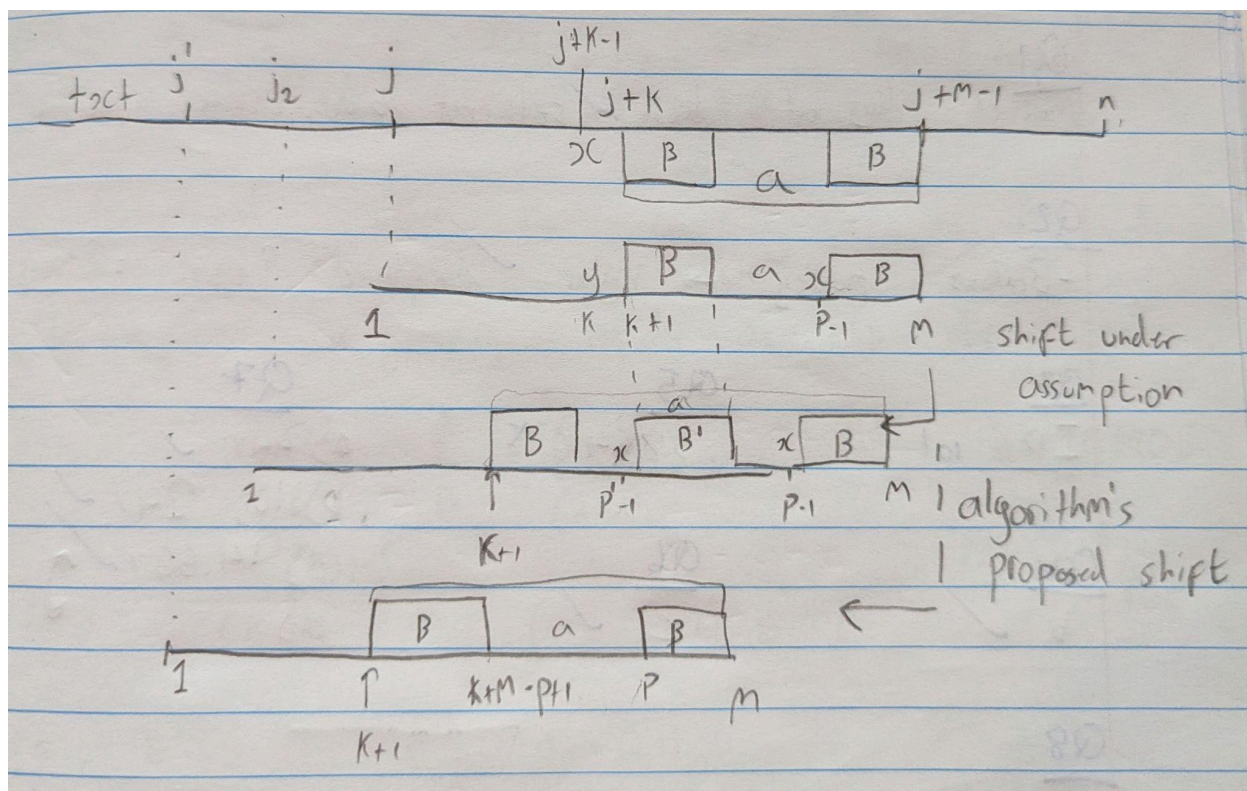
is safe (i.e., no valid matches are overlooked during shifting).

Consider a pattern, pat , of length m and a text, txt of length n . The algorithm begins by aligning at position $j = n - m$ such that the m^{th} character of pat , $pat[m]$ is compared to $txt[n]$. The algorithm scans right-to-left, $pat[1 \dots m]$ against $txt[j \dots j+m-1]$ (in 1 based indexing) until either a mismatch occurs at position k in the pattern or the entire pattern matches the $txt[j \dots j+m-1]$. This scan will produce some substring a , where $a = txt[j+k \dots j+m-1] = pat[k+1 \dots m]$ where $0 \leq k \leq m$. (If $k = m$, $|a| = 0$, therefore no information is gained and the algorithm shifts by 1, $j' = j - 1$). The shift-table in the algorithm defines for any i and character ' c ', $shift_table[i][c] = p$, where p is the leftmost index such that $pat[p \dots m] = pat[k+1 \dots k+m-p+1]$ and k is the pattern index of a mismatch $pat[k] \neq txt[j+k-1]$. The shift

Hence $i = k+1$.

By definition if $p > 0$, then p defines the endpoint of a prefix - suffix match of the substring a . Let this match be denoted as β . The proposed shift by the algorithm denotes a leftward shift $j' = j - (p - (k+1))$, such that the pattern will be aligned under the region $txt[j' \dots j' + m - 1]$.

To prove by contradiction Suppose this shift is unsafe. Then there exists a valid occurrence of the pattern at some position j_2 such that $j < j_2 < j'$. Since this is a valid occurrence, $pat[1 \dots m] = txt[j_2 \dots j_2 + m - 1]$. This is illustrated below:



At alignment j , we already know: $\text{txt}[j+k \dots j+m-1] = a$ and since $j' < j_2 < j$ it follows that the substring a overlaps with the occurrence at j_2 . So the suffix of the pattern at j_2 must match a , or $\text{pat}[p' \dots m] = a$ for some $p' < p$. Now consider the mismatch at position k , since $\text{pat}[k] \neq \text{txt}[j+k]$ the occurrence of a used for shifting must be preceded by a different character, but the occurrence at j_2 aligns a with a valid full match, and therefore its preceding character is consistent with the pattern. This gives a valid occurrence of a earlier than p satisfying the same condition. Since the match at j_2 is a full valid match, $\text{txt}[j+k] = \text{pat}[p'-1]$, meaning the mismatch character also equals the character immediately left of the overlap in the pattern. Therefore p' satisfies both condition 1 ($\text{pat}[p' \dots m] = \text{pat}[k+1 \dots k+m-p'+1]$) and condition 2 ($\text{pat}[p'-1] = \text{txt}[j+k]$), with $p' < p$. This directly contradicts the definition of p as the leftmost index satisfying both conditions simultaneously. Therefore no such j_2 can exist, and the shift is safe.

This contradicts the definition of p , where p is the leftmost index such that $\text{pat}[p \dots m] = a$. So there is no valid occurrence of j_2 with $j < j_2 < j'$.

Further, the proposed shift by this algorithm requires shifting by $p - (k+1)$ leftwards.

To verify this chosen value of p , note during construction $\text{shift_table}[r][\text{all } c \text{ in } \Sigma] = \text{shift_table}[m-L]$ and $\text{shift_table}[r][\text{pat}[p-i]] = p$ so we always insert smaller borders, then insert the current border. L is calculated as the size of a prefix suffix match, given by the LPS function of the KMP border chain. Applying this on a reversed pat provides all borders of every suffix $\beta = \text{Pat}[i \dots m-1]$, and so all valid borders for each character to the left are maintained providing the left-most value ' p '.

3. Characterize the worst-case time and space complexity of your implementation with justification.

Time-Complexity of Preprocessing:

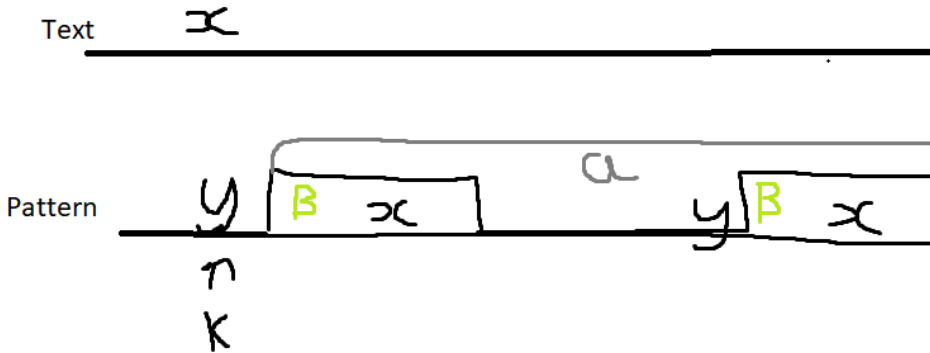
This implementation relies on computing the KMP Longest Prefix suffix table. This construction is done in $O(m)$ where m is the length of the pattern. This is optimised via dynamically updating $\text{lps}[i]$ from previously computed values. After initializing $\text{lps}[0]$, we note the value of the previous lps, say as L , then as we traverse the pattern we compare $\text{pat}[L]$ vs $\text{pat}[i]$. This gives three cases:

1. If $\text{Pat}[L] = \text{Pat}[i]$ we increment L by 1 and set $\text{pat}[i] = L$.
2. $\text{Pat}[L] \neq \text{Pat}[i]$ and $L = 0$, so $\text{LPS}[i] = 0$
3. $\text{Pat}[L] \neq \text{Pat}[i]$ but $L > 0$ For this case Since $\text{LPS}[i-1] = L$, we know $\text{Pat}[0 \dots L-1] = \text{Pat}[i-L \dots i-1]$ so we check $\text{lps}[\text{len}-1]$ giving us the longest suffix-prefix match at $\text{Pat}[0 \dots L-1]$, then we update L to this value and check if the next character, that is $\text{pat}[L] = \text{Pat}[i]$ giving either case 1 or 2.

Thus we only loop over the pattern once giving $O(m)$

This is also ran on the reversed pattern giving time complexity $O(M + M) = O(m)$

The construction of the shift table takes $O(m * |\Sigma|)$, This is done via looping over the rev_lps table defined earlier which is at most $O(m)$ and then for each character in the defined alphabet we have to write it as an entry in the table taking $O(|\Sigma|)$. Since we need to note all shorter but also correct shift borders for any character incase of a mismatched character in the largest suffix match, we can't just use the largest length defined in the LPS. This situation is shown in the diagram:



To do this we compute it dynamically, so we do not have to iterate over the LPS for each row. To do this we copy all shorter valid shift borders previously defined since these are all valid borders computed for each character match. That is to say, a smaller B where character x matches will be copied into the current shift table at $k+1$, for character = x, whilst character = y returns the larger B bounding the population of the table to $O(m * |\Sigma|)$

Time Complexity of Pattern Matching:

The time complexity of the matching part of the algorithm is $O(n)$. Where n is the length of the pattern.

With this preprocessed shift table the algorithm starts by aligning pattern and text at $j = n - m$.

Then as we match characters we define a matched region of text, when there is a mismatch at k, we search $\text{table}[k+1][c]$ where c is the mismatched character, this will give us p, the index to shift to so the shift = $p - k - 1$ or $m - k$ if the mismatched character does not exist in the matched region, as this means it is impossible to match any valid occurrences of the pattern.

This gives $j' = j - \text{shift}$ and we move our pattern leftwards to realign at this new j.

To save processing time after each shift I implemented the Galil rule found in more detail here:

<https://stackoverflow.com/questions/38206841/boyer-moore-galil-rule>

This notes after each shift how many characters to the right of the new alignment are already known to match. That is for the matched region of text, we know if we shift so that some B aligns with the text, we can start matching from $|B| + 1$, and retain the information of the matched region already, because of this each alignment computes comparisons with new characters in the pattern bounding the amount of comparisons to $O(n)$.

Summary:

Phase:

LPS table: $O(M)$

Shift table: $O(M * |\Sigma|)$

Search: $O(n)$

Total: $O(n + m * |\Sigma|)$

Where n is the length of text, m is the length of pattern and $|\Sigma|$ is the size of the alphabet.

Space complexity:

The algorithm creates a shift table, lps and rev_lps table and a result table.

The results table is bound to the number of matches, or $O(n)$ in worst case that every occurrence of text matches the pattern.

The Lps and Rev_LPs table are bound by $O(m)$ since they store the length of the pre-suffix match for each index in the pattern.

The shift table stores for each index and Character in the alphabet the shift position and is therefore bound by $O(m * |\Sigma|)$

This gives a final space complexity of **$O(n + m * |\Sigma|)$** .